

深圳大学实验报告

课程名称: 数据挖掘

实验项目名称: Python 编程

学院: 计算机与软件学院

专业: 计算机科学与技术（创新班）

指导教师: 王祎乐

报告人: 李文俊 学号: 2023150001 班级: 高性能

实验时间: 2026 年 04 月 02 日

实验报告提交时间: 2026 年 04 月 10 日

教务处制

实验目的与要求：

- 1、熟悉 python 基本语法。
- 2、掌握 python 编程基础。

试验环境：

1. 操作系统：Microsoft Windows 11
2. Python 版本：Python 3.13.5
3. 命令行环境：PowerShell 5.1.26100.7920
4. 开发工具：VS Code

实验内容及过程：

1. 题目 1：读取文件&字母排序

1.1 实验要求

有两个磁盘文件 A 和 B，各存放一行字母。要求读取两个文件中的内容，将两个字符串合并后按照字母顺序进行排序，并将排序后的结果输出到一个新的文件 C 中

1.2 程序设计

首先在脚本所在目录创建两个输入文件 test1.txt 和 test2.txt，分别在文件中写入一行字母字符串作为测试数据。程序运行时需要读取这两个文件的内容，并对数据进行处理。

程序实现步骤如下：

- (1) 使用 open() 函数分别读取两个文件中的首行内容；
- (2) 将两个字符串进行拼接，得到合并后的字符串；
- (3) 使用 Python 内置函数 sorted() 对字符串中的字符进行升序排序；
- (4) 将排序后的字符列表通过 "".join() 转换为字符串；
- (5) 将最终结果写入新的输出文件 test3.txt

1.3 程序实现

```
def merge_and_sort(file_a, file_b, file_c):  
    with open(file_a, "r", encoding="utf-8-sig") as fa:  
        a = fa.readline().strip()  
  
    with open(file_b, "r", encoding="utf-8-sig") as fb:  
        b = fb.readline().strip()  
  
    merged = a + b  
    result = "".join(sorted(merged))  
  
    with open(file_c, "w", encoding="utf-8") as fc:  
        fc.write(result)  
  
    print("merge done ->", file_c)  
    print("result:", result)  
  
if __name__ == "__main__":  
    merge_and_sort("test1.txt", "test2.txt",  
                  "test3.txt")
```

图 1 题目 1 源程序

1.4 运行结果

输入文件内容示例: test1.txt : bdca, test2.txt : gfed

程序运行命令: python exp1_q1.py

终端输出结果:

```
wenjun@Mac-3 实验一 Python编程 % python3 exp1_q1.py  
merge done -> test3.txt  
result: abcddefg
```

图 2 题目 1 输出结果

生成文件 test3.txt 中内容为: abcddefg

2. 题目 2：列表操作

2.1 实验要求

创建一个名为 `names` 的列表，并依次添加元素 `Lihua`、`Rain`、`Jack`、`Xiuxiu`、`Peiqi` 和 `Black`。然后分别完成以下操作：

- (1) 在 `Black` 前插入元素 `Blue`，在 `Black` 后插入元素 `White`；
- (2) 将列表中 `Xiuxiu` 替换为 "秀秀"；
- (3) 创建新列表 `[1,2,3,4,2,5,6,2]`，将其元素追加到 `names` 列表末尾，并取出 `names` 列表索引 2~10 的元素（步长为 2）

2.2 程序设计

首先编写函数 `build_names()` 创建并返回初始列表，使得每个操作都能从同一个初始列表开始，避免不同操作之间相互影响。

- (1) 在第一个操作中，通过 `index()` 方法找到 "Black" 在列表中的位置，然后使用 `insert()` 方法在其前后分别插入 "Blue" 和 "White"
- (2) 在第二个操作中，通过 `index()` 找到 "Xiuxiu" 的位置，并直接进行元素替换
- (3) 在第三个操作中，创建一个新的整数列表，并使用 `extend()` 方法将该列表的元素追加到 `names` 列表末尾。随后利用列表切片操作 `names[2:11:2]` 提取指定范围的元素

2.3 程序实现

```
def build_names():
    return ["Lihua", "Rain", "Jack", "Xiuxiu",
            "Peiqi", "Black"]

def operation_1():
    names = build_names()
    black_index = names.index("Black")
    names.insert(black_index, "Blue")
    names.insert(black_index + 2, "White")
    print("Operation 1:", names)

def operation_2():
    names = build_names()
    xiuxiu_index = names.index("Xiuxiu")
    names[xiuxiu_index] = "秀秀"
    print("Operation 2:", names)

def operation_3():
    names = build_names()
    extra_list = [1, 2, 3, 4, 2, 5, 6, 2]
    names.extend(extra_list)

    print("Operation 3 (extended):", names)

    picked = names[2:11:2]
    print("Operation 3 (index 2-10, step=2):",
          picked)

if __name__ == "__main__":
    operation_1()
    operation_2()
    operation_3()
```

图 3 题目 2 源程序

2.4 运行结果

程序运行后输出结果如下：

```
wenjun@Mac-3 实验一 Python编程 % python3 exp1_q2.py
Operation 1: ['Lihua', 'Rain', 'Jack', 'Xiuxiu',
'Peiqi', 'Blue', 'Black', 'White']
Operation 2: ['Lihua', 'Rain', 'Jack', '秀秀',
'Peiqi', 'Black']
Operation 3 (extended): ['Lihua', 'Rain', 'Jack',
'Xiuxiu', 'Peiqi', 'Black', 1, 2, 3, 4, 2, 5, 6, 2]
Operation 3 (index 2-10, step=2): ['Jack', 'Peiqi',
1, 3, 2]
```

图 4 题目 2 输出结果

运行结果表明，程序能够正确完成列表插入、替换、扩展以及切片操作。

3. 题目 3：字典库存统计

3.1 实验要求

定义一个字典，用于表示玩家拥有的物品清单。字典的键为物品名称，值为物品数量。编写函数 `displayInventory()`，用于打印每种物品的数量及名称，并统计所有物品的总数量。

3.2 程序设计

- (1) 首先创建一个字典 `inventory`，用于存储不同物品及其数量
- (2) 随后编写函数 `displayInventory()`，该函数接收字典作为参数，函数内部首先输出标题 "Inventory:"，然后通过 `for` 循环遍历字典中的键值对。每次循环输出当前物品的数量和名称，并将物品数量累加到变量 `total_items` 中
- (3) 循环结束后输出所有物品的总数

3.3 程序实现

```
def displayInventory(inventory):
    print("Inventory:")
    total_items = 0

    for item_name, item_count in inventory.items():
        print(f"{item_count} {item_name}")
        total_items += item_count

    print(f"Total number of items: {total_items}")

if __name__ == "__main__":
    stuff = {
        "arrow": 12,
        "gold coin": 42,
        "rope": 1,
        "torch": 6,
        "dagger": 1,
    }

    displayInventory(stuff)
```

图 5 题目 3 源程序

3.4 运行结果

程序运行结果如下：

```
wenjun@Mac-3 实验一 Python编程 % python3 exp1_q3.py
Inventory:
12 arrow
42 gold coin
1 rope
6 torch
1 dagger
Total number of items: 62
```

图 6 题目 3 输出结果

4. 题目 4：读取文件&字母排序

4.1 实验要求

给定两个非空链表，分别表示两个非负整数。每个链表中的数字按逆序存储，并且每个节点只存储一位数字。要求编写程序计算两个整数之和，并以相同的链表形式返回结果。

4.2 程序设计

- (1) 首先定义链表节点类 `ListNode`，用于表示链表中的节点结构
- (2) 随后编写函数 `addTwoNumbers()` 实现链表加法运算：程序通过循环同时遍历两个链表节点，并逐位计算对应数字之和，同时使用变量 `carry` 保存进位。当某一位的结果大于等于 10 时，更新进位并将当前位的结果存储到新节点中。计算过程一直持续到两个链表遍历结束且没有进位为止。
- (3) 还编写辅助函数 `build_list()` 用于根据数组创建链表，以及 `to_list()` 用于将链表转换为列表方便输出

4.3 程序实现

```

class ListNode(object):
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def addTwoNumbers(l1, l2):
    dummy = ListNode(0)
    current = dummy
    carry = 0

    while l1 is not None or l2 is not None or carry:
        x = l1.val if l1 is not None else 0
        y = l2.val if l2 is not None else 0

        total = x + y + carry
        carry = total // 10
        current.next = ListNode(total % 10)
        current = current.next

        if l1 is not None:
            l1 = l1.next
        if l2 is not None:
            l2 = l2.next

    return dummy.next

def build_list(values):
    dummy = ListNode(0)
    current = dummy
    for value in values:
        current.next = ListNode(value)
        current = current.next
    return dummy.next

def to_list(head):
    values = []
    while head is not None:
        values.append(head.val)
        head = head.next
    return values

if __name__ == "__main__":
    l1 = build_list([2, 4, 3])
    l2 = build_list([5, 6, 4])

    result = addTwoNumbers(l1, l2)
    print(to_list(result)) # [7, 0, 8]

```

图 7 题目 4 源程序

4.4 运行结果

输入: l1 = [2,4,3], l2 = [5,6,4] 表示整数: 342 + 465

终端输出结果: [7, 0, 8] 结果表示整数 807, 说明程序能够正确实现链表表示的整数加法运算

```

wenjun@Mac-3 实验一 Python编程 % python3 exp1_q4.py
[7, 0, 8]

```

图 8 题目 4 输出结果

5. 题目 5: 最长无重复子串

5.1 实验要求

给定一个字符串，要求找出其中不含有重复字符的最长子串的长度。例如字符串 "abcabcbb" 的最长无重复子串为 "abc"，长度为 3。

5.2 程序设计

- (1) 本题采用滑动窗口算法进行求解，程序使用两个变量 left 和 right 表示当前窗口的左右边界，同时使用字典 last_pos 记录每个字符最近一次出现的位置
- (2) 在遍历字符串过程中，如果当前字符已经在窗口中出现过，则将窗口左边界移动到该字符上一次出现位置的下一位，从而保证窗口中始终不包含重复字符
- (3) 在每次循环中更新当前窗口长度，并记录最大值

5.3 程序实现

```
def lengthOfLongestSubstring(s):  
    left = 0  
    max_len = 0  
    last_pos = {}  
  
    for right, ch in enumerate(s):  
        if ch in last_pos and last_pos[ch] >= left:  
            left = last_pos[ch] + 1  
  
        last_pos[ch] = right  
        max_len = max(max_len, right - left + 1)  
  
    return max_len  
  
if __name__ == "__main__":  
    s1 = "abcabcbb"  
    s2 = "bbbbbb"  
  
    print(f"s = {s1}, length =  
    {lengthOfLongestSubstring(s1)}")  
    print(f"s = {s2}, length =  
    {lengthOfLongestSubstring(s2)}")
```

图 9 题目 5 源程序

5.4 运行结果

测试输入：s = "abcabcbb", s = "bbbbbb"

终端输出结果：

```
wenjun@Mac-3 实验一 Python编程 % python3 exp1_q5.py  
s = abcabcbb, length = 3  
s = bbbbbb, length = 1
```

图 10 题目 5 输出结果

结果表明程序能够正确计算字符串中最长无重复子串的长度。

实验收获：

通过本次实验，我进一步熟悉了 Python 的基本语法以及常见的数据结构和程序设计方法。在完成实验题目的过程中，实践了文件读写、字符串处理、列表与字典操作等基础功能，对 Python 中常用的数据类型及其相关方法有了更加直观的认识。同时，通过编写多个独立的小程序，也加深了对函数封装和程序结构组织的理解，使代码逻辑更加清晰、易于维护。

在实验过程中，还接触并实现了一些具有一定算法思想的问题，例如利用链表结构实现整数相加，以及通过滑动窗口方法求解字符串中最长无重复子串的问题。这些内容让我体会到在解决实际问题时，合理选择数据结构和算法思路的重要性，也进一步提高了对程序运行效率和逻辑设计的认识。

在编写和调试程序的过程中，也遇到了一些问题。例如在进行文件读取时需要注意换行符的处理，以及在列表操作和字符串处理过程中要正确理解函数的返回值和使用方法。通过查阅资料和反复调试程序，最终解决了这些问题，使程序能够正确运行。通过本次实验，不仅巩固了 Python 编程基础，也提升了独立分析问题和解决问题的能力，为后续课程的学习打下了良好的基础。

指导教师批阅意见:

成绩评定:

指导教师签字:

年 月 日

备注:

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。